The TipTap editor serves as the cornerstone of our document creation workflow, providing a professional, enterprise-grade rich text editing experience that transforms how users compose RTI requests. Unlike traditional contentEditable implementations that rely on the deprecated document.execCommand API, TipTap leverages ProseMirror's robust document model to maintain a clean, predictable state representation of everything users type. This architectural decision ensures that formatting, structure, and custom elements like variables and page breaks remain consistent throughout the editing session, regardless of how complex the document becomes. The editor seamlessly integrates with our React application through custom hooks and components, making it feel native to the overall user experience while providing capabilities that would be extremely difficult to implement from scratch.

At the heart of TipTap lies a sophisticated document state management system that represents every piece of content as a structured tree of nodes and marks. Nodes represent block-level elements such as paragraphs, headings, lists, and our custom page break elements, while marks handle inline formatting like bold, italic, underline, and text alignment. This separation between structure and style enables precise control over document semantics, ensuring that when users apply bold formatting to text, it's stored as a mark on a text node rather than as inline HTML styling. The document state is immutable, meaning every change creates a new state snapshot, which enables perfect undo/redo functionality and makes the editor incredibly reliable even with complex nested formatting.

Our implementation extends TipTap's core functionality with two critical custom nodes that address the specific requirements of RTI document creation. The PageBreak node renders as a visual dashed line in the editor but serializes to a special marker in markdown, which the PDF generation engine detects to create new physical pages in the final document. The VariablePill node represents dynamic placeholders like Dilini Fernando or 2026-06-17 as styled, atomic pill elements that users cannot edit internally, ensuring variable integrity. Both custom nodes leverage TipTap's NodeViewRenderer system to render as full React components, complete with delete buttons and interactive elements, demonstrating how TipTap seamlessly bridges the gap between document state and React's component model.

One of the most powerful features of our SmartEditor implementation is the robust HTML-to-Markdown-to-HTML conversion pipeline that ensures content remains clean and portable regardless of its origin. When users paste content from external sources like Google Docs, Microsoft Word, or web pages, the raw HTML often contains hundreds of lines of garbage styles, inline formatting artifacts, and browser-specific quirks that would break the editor if inserted directly. Our pipeline first converts this messy HTML to clean, standardized markdown using a recursive DOM walker that handles all element types, then converts that markdown back to pristine HTML with

semantic tags and proper structure. This normalization process ensures that content from any source appears consistently in the editor and produces reliable PDF output.

The paste handling system in our SmartEditor goes far beyond simple text insertion, implementing a multi-step cleaning and normalization process that protects document integrity while preserving meaningful formatting. When users initiate a paste operation, the handlePaste function intercepts the event, reads both HTML and plain text versions from the clipboard, and applies a sophisticated cleaning algorithm that removes Google Docs wrapper tags, strips Word's mso-specific styles, eliminates empty formatting spans, and converts inline CSS styles to semantic HTML equivalents. This cleaning process is particularly important for variables and page breaks, which must survive the paste operation intact, and for maintaining consistent formatting across documents created by different users with different source applications.

The variable system integrated with TipTap enables users to create dynamic document templates that automatically populate with real data when generating PDFs or finalizing requests. When users insert variables like Dilini Fernando or Ministry of Health and Nutrition into the editor, they appear as visually distinct pill elements that display the variable's friendly name while storing its code identifier as data attributes. The getVariableValues function extracts actual values from sender and receiver objects, mapping each variable code to its corresponding data field through a flexible normalization system that handles different spacing, case, and formatting variations. This means users can write Dilini Fernando, Dilini Fernando, or Dilini Fernando and the system will correctly replace it with the applicant's actual name from the database.

The PDF generation engine, powered by jsPDF, seamlessly integrates with TipTap's content model to produce professional, print-ready documents that respect all formatting and page breaks defined in the editor. The generateRTIPDF function begins by replacing all variables with actual data values, stripping HTML artifacts from variable pills, and normalizing text alignment styles into a consistent format. As it processes each line of the final markdown content, it maintains a cursor position on the PDF page and performs sophisticated line wrapping that considers font styles, text sizes, and available width. When the parser encounters the marker generated by our PageBreak node, it calls doc.addPage() to create a new physical page, ensuring that users have precise control over document pagination.

The PDF rendering system implements a sophisticated parsing pipeline that converts formatted markdown text into individual tokens with associated styling information, enabling precise control over how each word appears on the page. The parseInlineSegments function uses regular expressions to identify formatting markers like **bold** , *italic* , ***bolditalic*** , and underline , generating an array of

segment objects that track the formatting state of each text chunk. These segments are then converted to tokens, splitting text on word boundaries while preserving whitespace as separate tokens to maintain proper word spacing and indentation. The token wrapping function then groups these formatted tokens into lines based on the available page width, ensuring that bold, italic, and underline formatting is preserved even when text wraps to a new line.

Our PDF generator supports all four text alignments - left, center, right, and justified - with sophisticated rendering logic that produces professional-looking documents regardless of alignment choice. For left, center, and right alignment, the getAlignX function calculates the starting X position for each line based on the line width and the chosen alignment, centering text or pushing it to the right edge as appropriate. For justified alignment, the renderRowJustified function calculates the total width of all words in a line, determines how much extra space needs to be distributed between words, and evenly spaces each word across the full available width. This implementation handles the complexity of mixed formatting within a single line, ensuring that bold and italic words maintain their styling while participating in the justification calculation.

The complete integration of TipTap with our variable system, PDF generation, and paste handling has transformed the RTI request creation process from a frustrating, bug-prone experience into a smooth, professional workflow that users can trust. Users can now compose complex documents with proper formatting, insert page breaks precisely where needed, include dynamic variables that auto-populate with correct data, and paste content from any source without worrying about broken formatting or security issues. The consistency between what users see in the editor and what appears in the final PDF builds confidence and reduces the need for manual corrections, while the maintainable, modular codebase ensures we can continue adding features like tables, images, and more complex layouts in the future. This investment in a professional editing experience ultimately saves users time, reduces errors, and produces higher-quality RTI requests that reflect well on the organization.